Estructuras de Datos

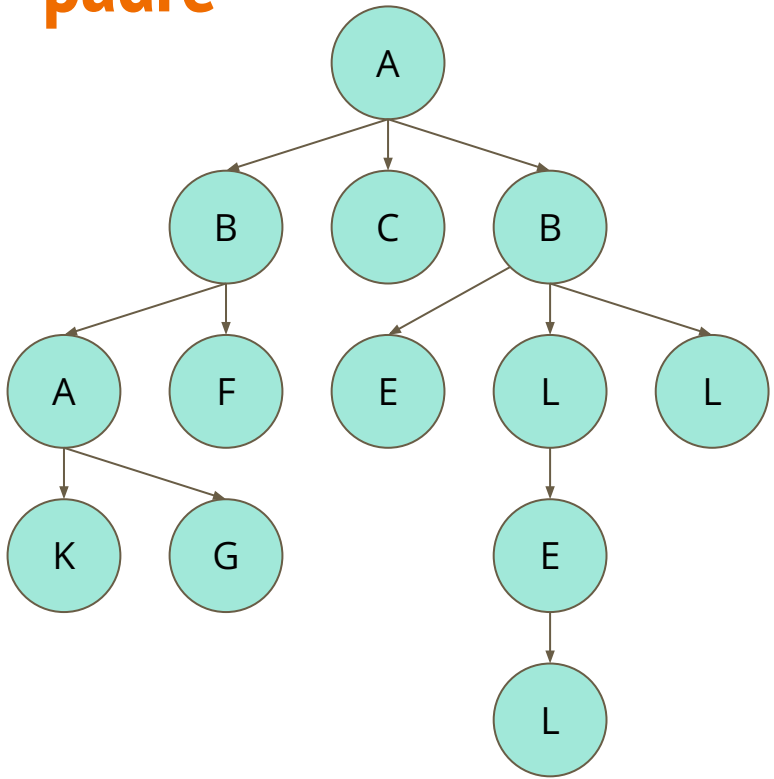
2026
Primer Cuatrimestre

REPASO Y CONT.

Representación de lista de hijos + padre

Arbol<E> implements Tree<E>
raiz: TNodo<E> size: integer

TNodo<E> implements Position<E>
elem: E padre: TNodo<E> hijos: PositionList<TNodo<E>>

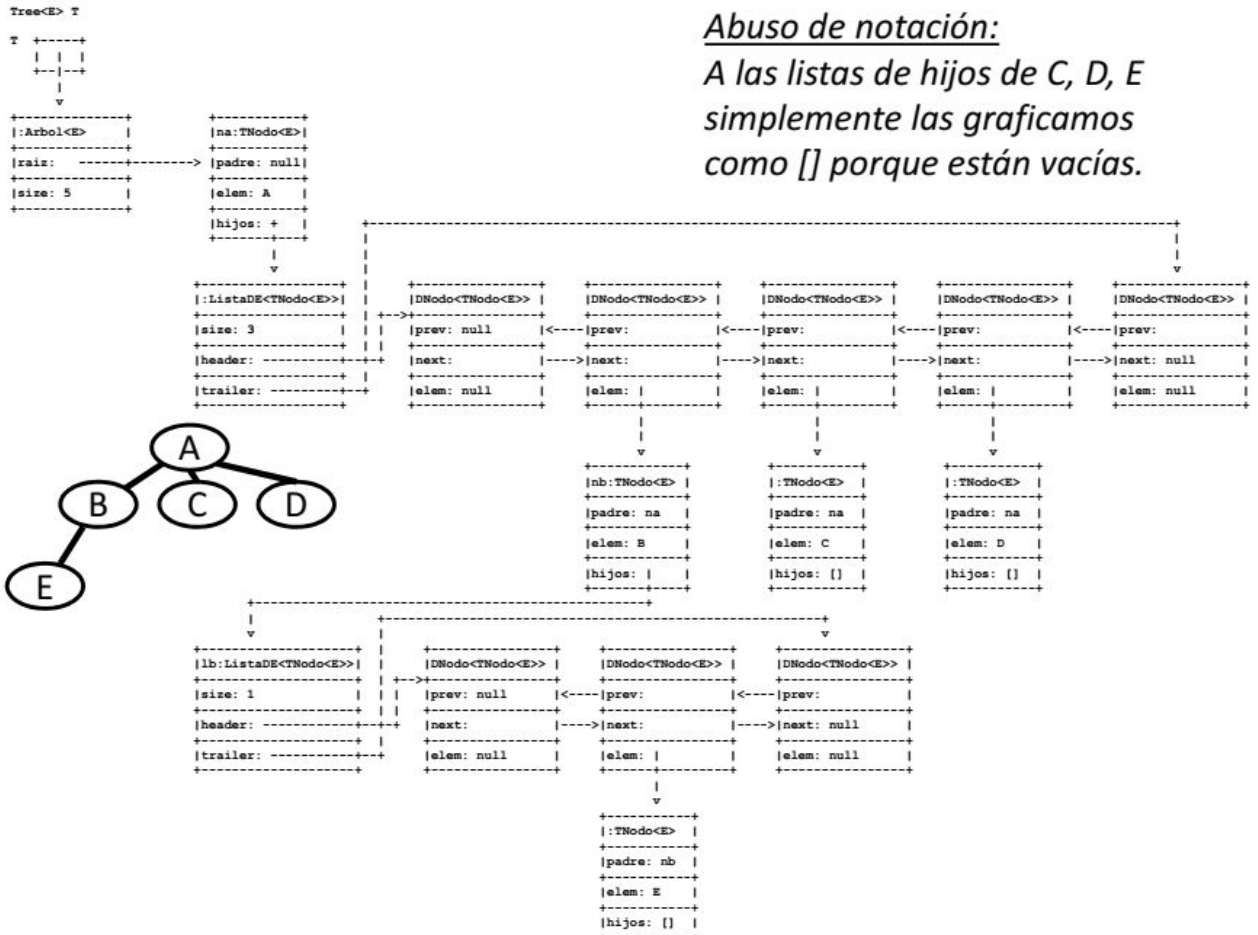


Aclaraciones de la clase pasada

- Diferencias entre las posiciones de listas y las posiciones de nodo
  - Usar el remove() de TDA lista para hacer un remove del TDA árbol
  - Lo mismo ocurre con los iteradores (de elementos o posiciones)
- Seteo de elementos como null al ser eliminados

```
private TNodo<E> checkPosition (Position<E> p) {
	TNodo<E> resultado = null;
	if (p == null) {
		throw new InvalidPositionException("Posición nula.");
	}
	if (this.isEmpty()) {
		throw new InvalidPositionException("Posición inválida");
	}
	try {
		resultado = (TNodo<E>) p;
	} catch (ClassCastException e) {
		throw new InvalidPositionException("Posición inválida");
	}
	return resultado;
}
```

Diagrama de objetos (representación GT)



Abuso de notación:

A las listas de hijos de C, D, E simplemente las graficamos como [] porque están vacías.

iterator()

Estrategia:

- Tengo que devolver un iterador de elementos/rótulos → lista a retornar
- Puedo apoyarme en el iterador de posiciones que me recorre el árbol
- Le pido los elementos al iterador de posiciones del árbol y los agrego a la lista a retornar

iterator()

```
public Iterator iterator() {
        PositionList l = new ListaDoblementeEnlazada();
        for (Position p : positions() )
                l.addLast(p.element() );
        return l.iterator();
}
```

iterator()

```
public Iterator iterator() {
        PositionList l = new ListaDoblementeEnlazada();
        for (Position p : positions() )
                l.addLast(p.element() );
        return l.iterator();
}
```

c_1

positions: $c_2 n$. Bucle repite n veces

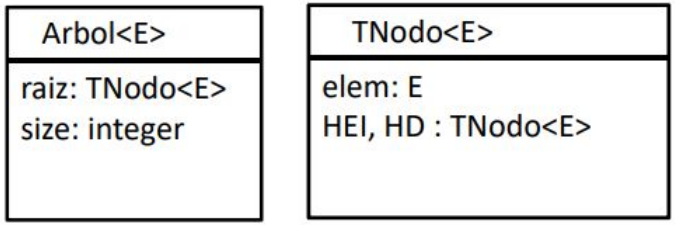
c_3

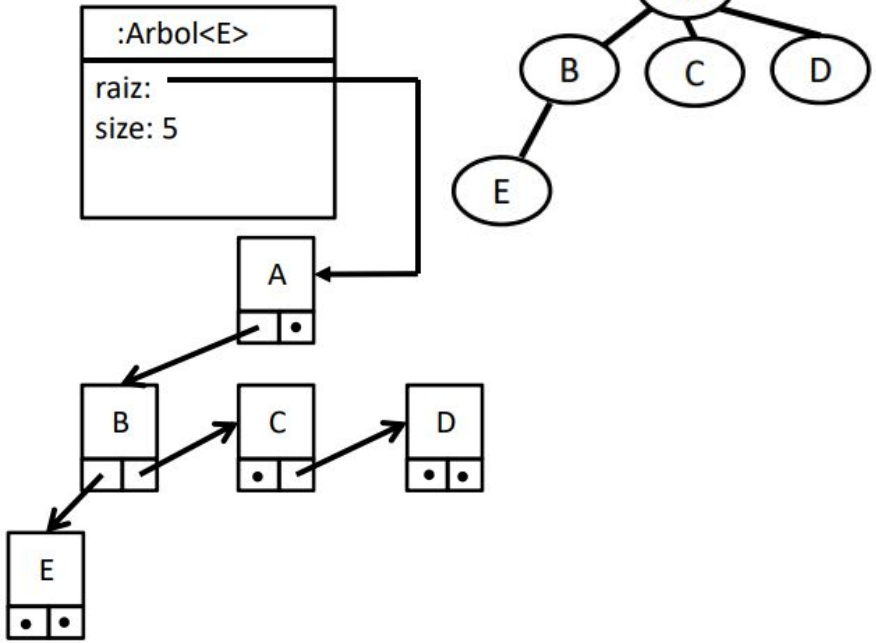
c_4

$T_{iterator}(n) = c_1 + c_2 n + c_3 n + c_4 \rightarrow O(2n) = O(n)$

Otras representaciones:
Hijo extremo izquierdo - hermano derecho

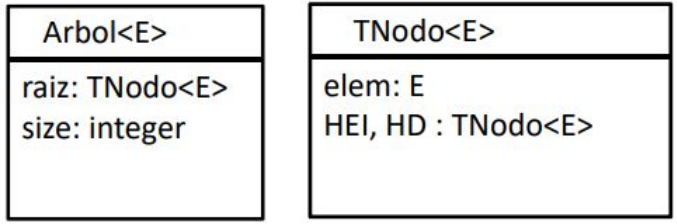
- Cada nodo conoce:
  - su rótulo/elemento
  - su hijo extremo izquierdo
  - su hermano derecho
  - (se puede combinar con la representación del padre de ser necesario)



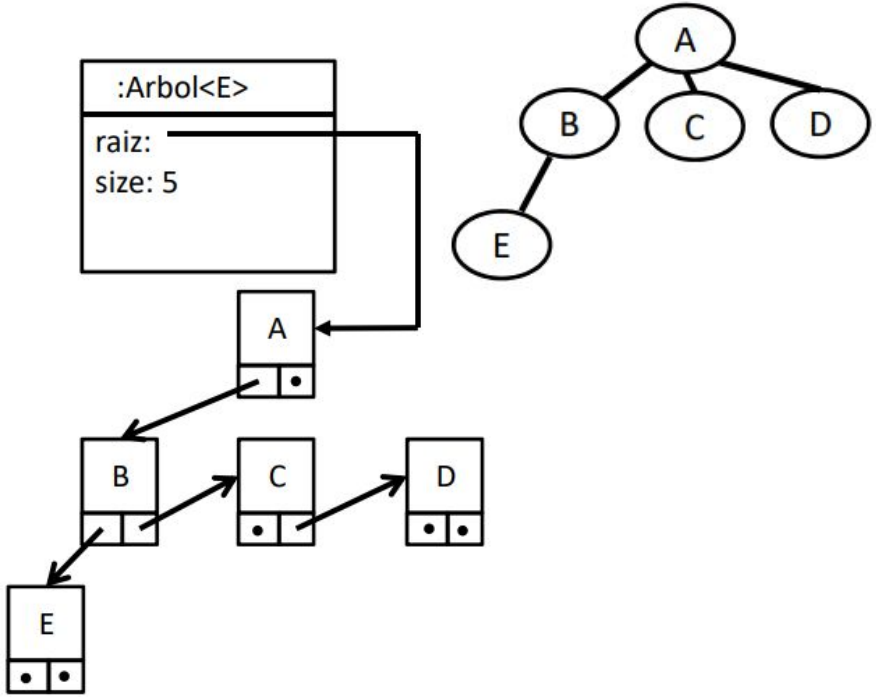


Notar que no hay positionLists!

Recorrido preorden (HEI-HD)



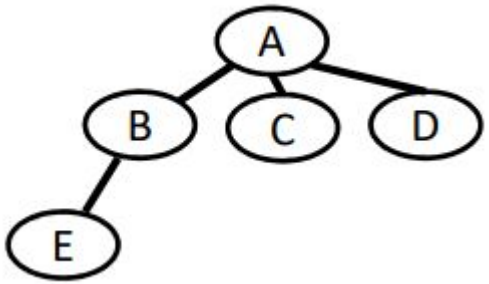

```
public void preorden() {
        pre( raiz );
}
// v representa la raíz del subárbol listado
actualmente
private void pre( TNodo<E> v ) {
        System.out.println( v.getElem() );
        TNodo<E> h = v.getHEI();
        while( h != null ) {
                pre( h );
                //avanzo al siguiente hijo
                que es el hno de h
                h = h.getHD();
        }
}
```



Representación con arreglos de HEI-HD

- Útil cuando el lenguaje no tiene soporte de memoria dinámica (e.g. ensamblador) o dispositivos edge
- Representación:
  - Arreglo de rótulos,
  - arreglo de padres,
  - arreglo de HEI,
  - arreglo de HD,
  - cada nodo se representa con un cursor (una posición del arreglo),
    - 0 indica el nodo nulo, la posición 0 del arreglo no se usa.
  - Posición del nodo raíz y el tamaño del árbol

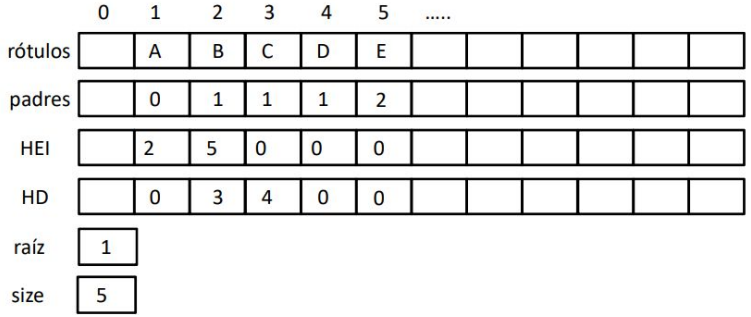
Representación con arreglos de HEI-HD



	0	1	2	3	4	5	.....					
rótulos		A	B	C	D	E						
padres		0	1	1	1	2						
HEI		2	5	0	0	0						
HD		0	3	4	0	0						

raíz	1
size	5

Recorrido preorden - representación con arreglos

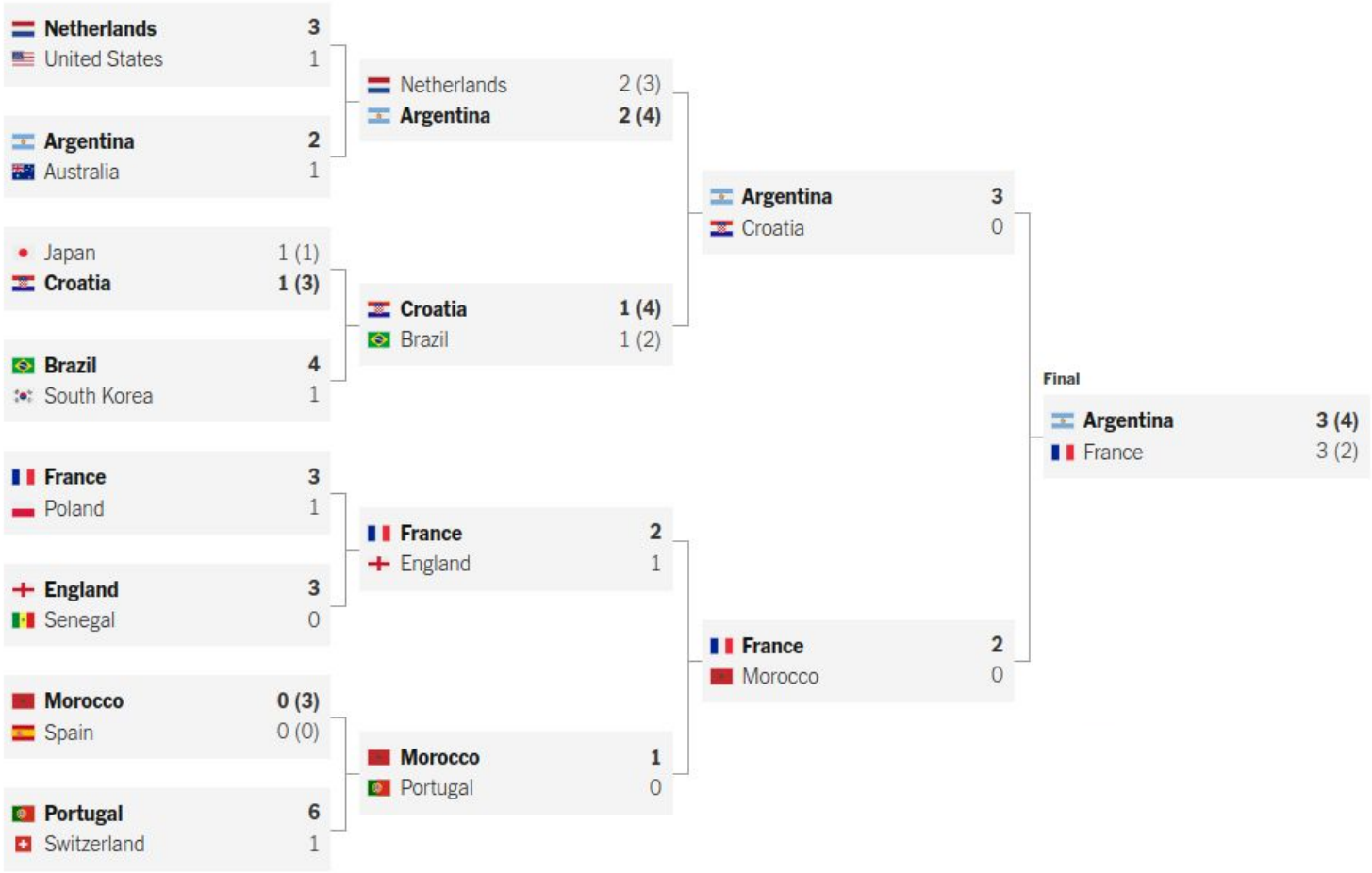


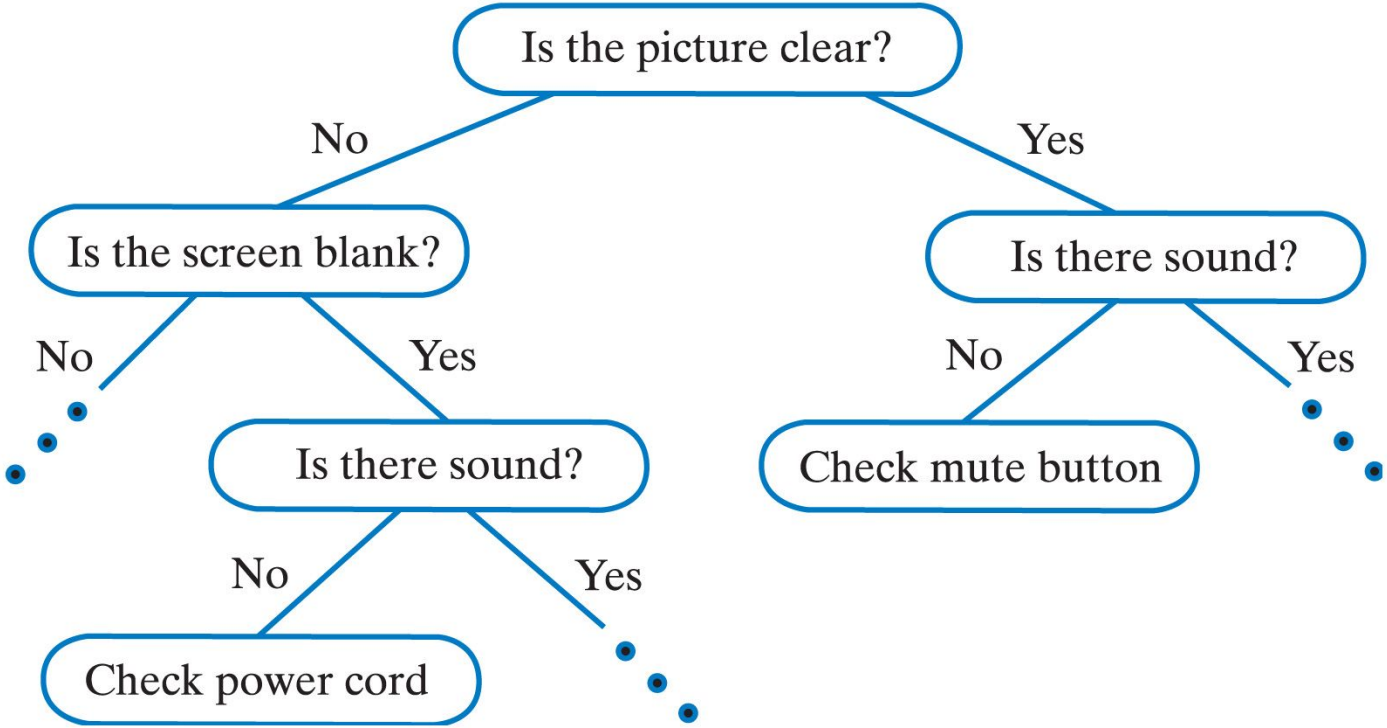
	0	1	2	3	4	5	.....
rótulos		A	B	C	D	E	
padres		0	1	1	1	2	
HEI		2	5	0	0	0	
HD		0	3	4	0	0	

raíz	1
size	5

```
public void preorden() {
        pre( raiz );
}
// v representa el cursor de la raíz del subárbol listado actualmente
private void pre( int v ) {
        System.out.println( rotulos[v] );
        int h = HEI[v]; // h representa el hijo actual procesado
        while( h != 0 ) { // mientras h no sea nulo hacer
                pre( h ); // preorden de h
                h = HD[h]; // avanzar al siguiente hijo h (que es el hermano de h)}
}
```

UN TIPO ESPECIAL DE ÁRBOL


Netherlands 3
United States 1
Argentina 2
Australia 1
Japan 1 (1)
Croatia 1 (3)
Brazil 4
South Korea 1
France 3
Poland 1
England 3
Senegal 0
Morocco 0 (3)
Spain 0 (0)
Portugal 6
Switzerland 1
Netherlands 2 (3)
Argentina 2 (4)
Croatia 1 (4)
Brazil 1 (2)
France 2
England 1
Morocco 1
Portugal 0
Argentina 3
Croatia 0
France 2
Morocco 0
Final
Argentina 3 (4)
France 3 (2)


Is the picture clear?
No
Yes
Is the screen blank?
Is there sound?
No
Yes
No
Yes
Is there sound?
Check mute button
No
Yes
Check power cord

ÁRBOLES BINARIOS

Árboles Binarios

Un árbol binario es un árbol ordenado que cumple:

1) Cada nodo tiene a lo sumo dos hijos (0, 1 ó 2)

2) Cada nodo distinto a la raíz es o bien hijo izquierdo o hijo derecho

3) El hijo izquierdo precede al hijo derecho en el orden de los hijos de un nodo

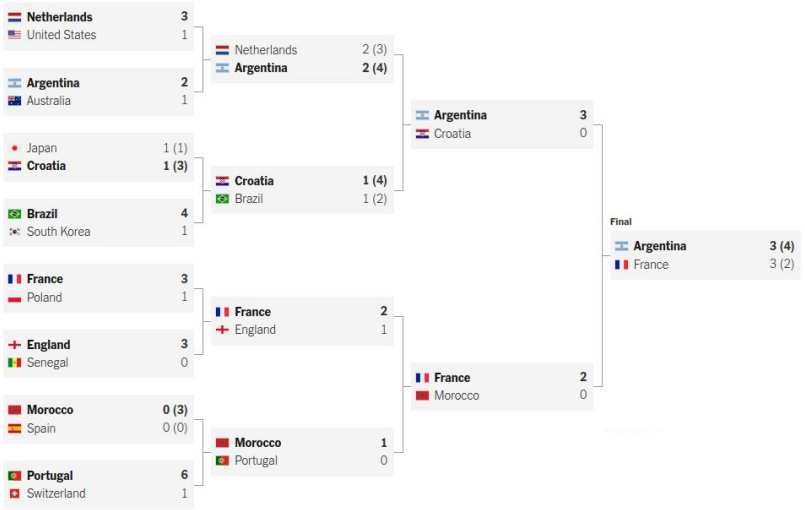
Árboles Binarios

El subárbol que tiene como raíz al hijo izquierdo se llama subárbol izquierdo.

El subárbol que tiene como raíz al hijo derecho se llama subárbol derecho.

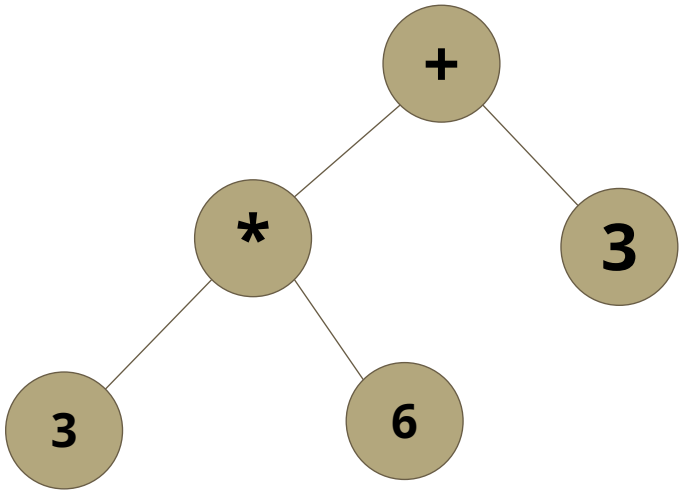
En un árbol binario propio, cada nodo tiene 0 o dos hijos (el libro también lo llama *full BT*). Si un árbol binario no es propio, entonces es impropio.

Árboles Binarios

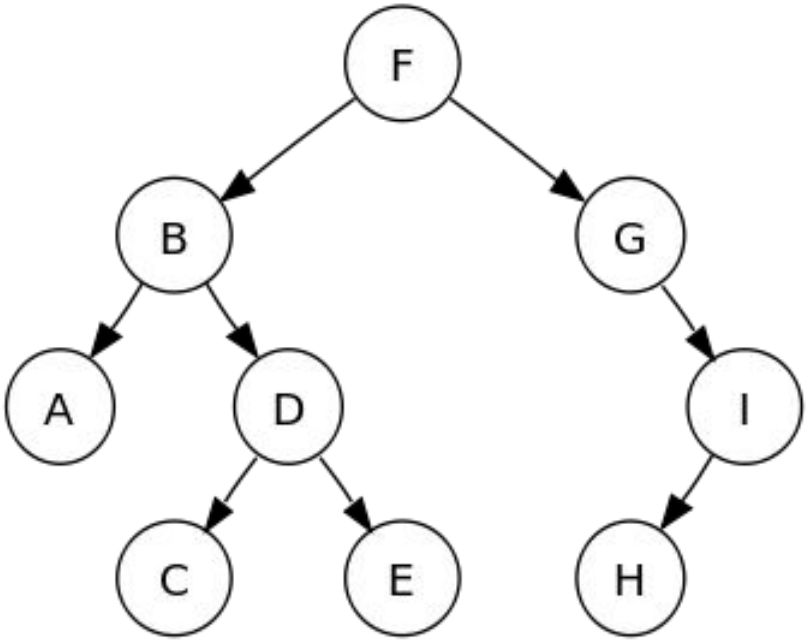


Árbol Binario Propio

Árbol binario propio

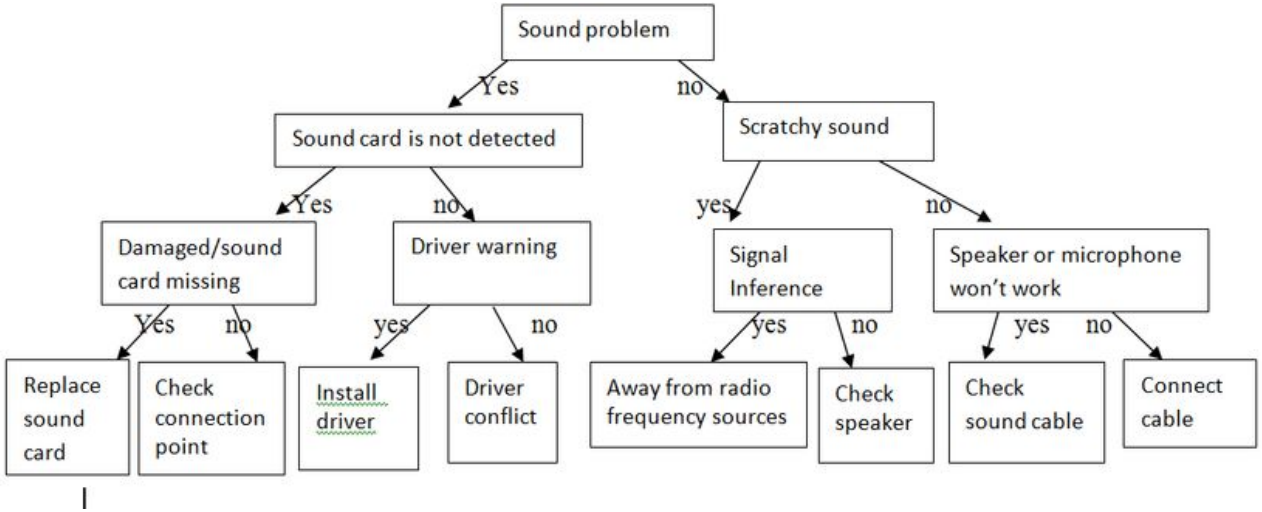


Árboles Binarios



Árbol Binario Impropio

Árboles Binarios para representar Decisiones



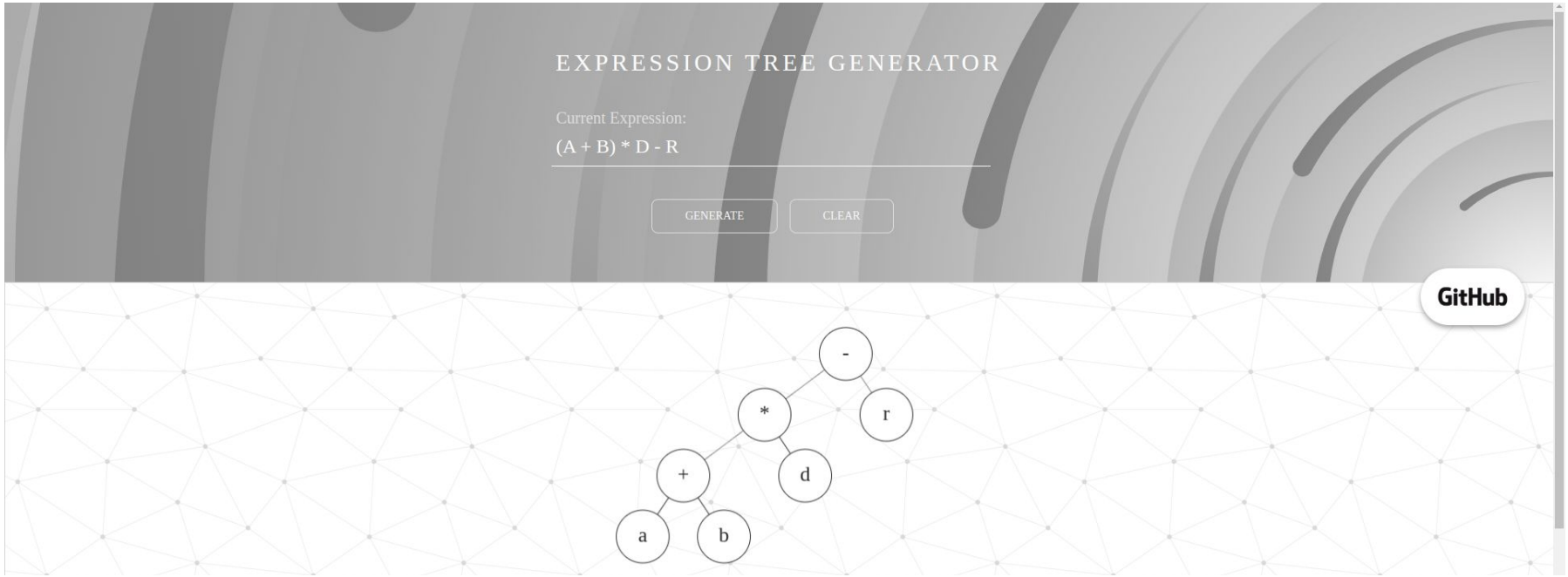
Árboles Binarios para representar Decisiones

Se desea representar los resultados asociados a un conjunto de preguntas con respuesta “Si” o “No”.

Cada nodo interno se asocia con una pregunta. Se comienza en la raíz y con cada pregunta se va a la izquierda o a la derecha dependiendo de si la respuesta a la pregunta es “si” o “no”.

Cuando se llega a una hoja, se tiene un resultado al cual se llega partir de las respuestas dadas en los ancestros de la hoja.

Árboles Binarios para representar expresiones aritméticas



https://lnogueir.github.io/expression-tree-gen/

Árboles Binarios para representar expresiones aritméticas



Árboles Binarios para representar expresiones aritméticas

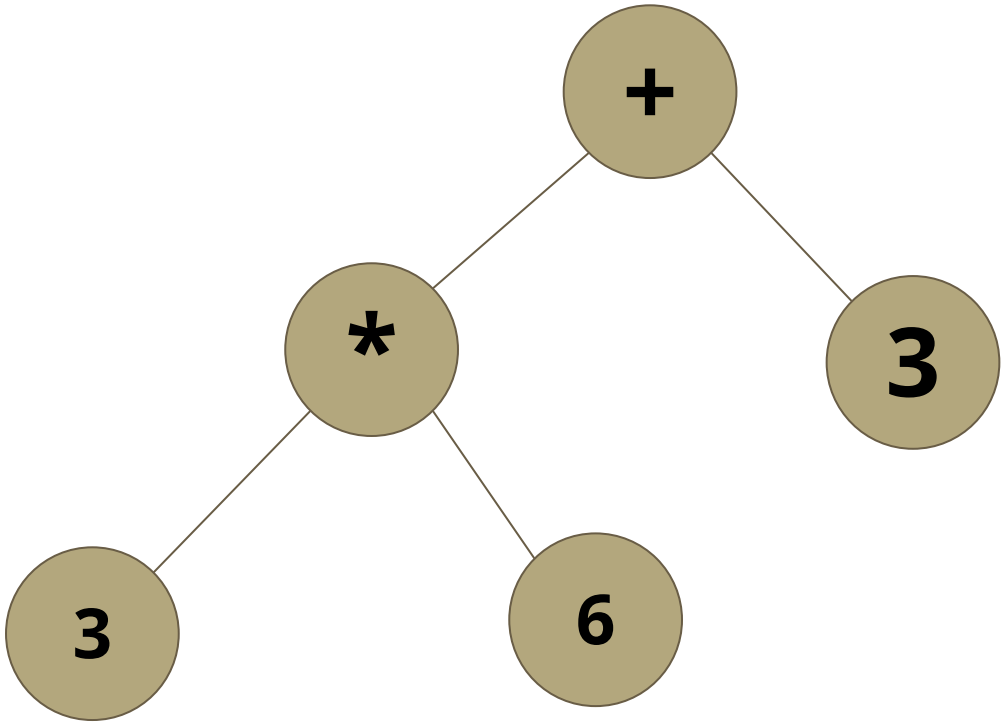
Los árboles binarios que representan operaciones aritméticas (AB de expresión) tienen la característica de que sus nodos internos son operadores y los nodos externos son operandos.

Dependiendo del tipo de recorrido que se haga sobre un AB de expresión se pueden obtener las distintas notaciones:

-Preorden: notación prefija

-Posorden: notación posfija

-Inorden: notación infija


+
*
3
3
6

Árboles Binarios - definición recursiva

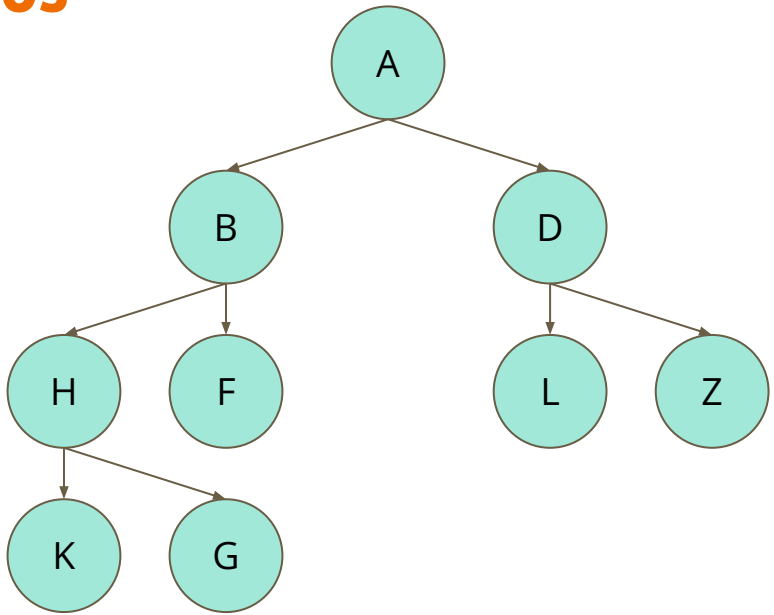
Un árbol binario T es o bien vacío, o consiste de:

- Un nodo r, llamado la raíz de T, que contiene un rótulo (o elemento)
- Un árbol binario, llamado el subárbol izquierdo de T
- Un árbol binario, llamado el subárbol derecho de T

Árboles Binarios - Nodos enlazados

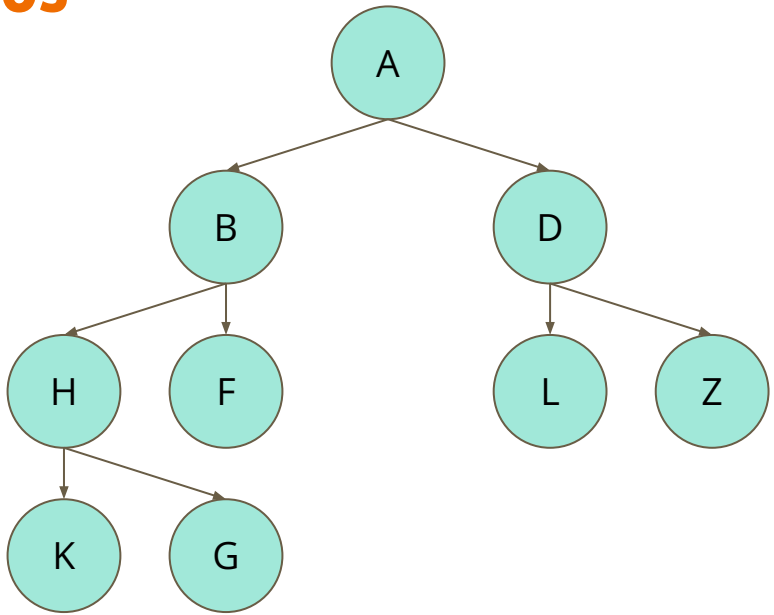
Arbol<E> implements Tree<E>
raiz: TNodo<E> size: int

TNodo<E> implements Position<E>
elem: E hijos: PositionList<TNodo<E>> padre:TNodo<E>



Árboles Binarios - Nodos enlazados

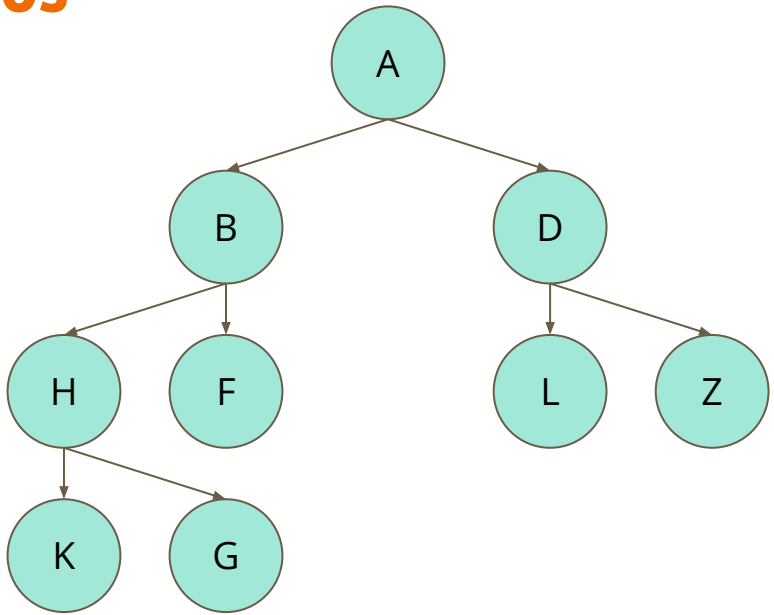
BTNodo<E> implements Position<E>
elem:E padre:BTNodo<E> hijoI:BTNodo<E> hijoD:BTNodo<E>



Árboles Binarios - Nodos enlazados

BinaryTree<E> extends Tree<E>

ArbolBinario<E> implements BinaryTree<E>
raiz: BTNodo<E> size: int



Árboles Binarios - Nodos enlazados

El libro de la cátedra plantea a los AB como un caso especial de los árboles generales (BinaryTree<E> extends GTTree<E>). Otros autores plantean al AB como algo separado del Tree.

Además, el libro propone el uso de la interface BTPosition<E>, la cual a fines prácticos resulta siendo equivalente a usar Position<E>.

Árboles Binarios - Nodos enlazados

Nosotros utilizaremos la implementación mencionada al comienzo:

- Una clase BTNodo que implemente a la interface Position;
- Una interface BinaryTree que extienda a la interface Tree;
- Una clase ArbolBinario que implemente la interface BinaryTree;

Arboles binarios - Interface BinaryTree

```
public interface BinaryTree<E> extends Tree<E>{
    public Position<E> right(Position<E> v)
    public Position<E> left(Position<E> v)
    public boolean hasLeft(Position<E> v)
    public boolean hasRight(Position<E> v)
    public Position<E> addLeft(Position<E> v, E r)
    public Position<E> addRight(Position<E> v, E r)
    public void attach(Position<E> r, BinaryTree<E> T1, BinaryTree<E> T2)
```

Clase BTNodo

```
public class BTNodo<E> implements Position<E> {

        private E element;

        private BTNodo<E> left, right, parent;

        public BTNode( E element, BTNodo<E> left, BTNodo<E> right, BTNodo<E>
parent ) {
                this.element = element; this.left = left;
                this.right = right; this.parent = parent; }

        // setters y getters para element, left, right y parent.

}
```

Break

Implementación de árbol binario

```
public class ArbolBinarioEnlazado<E> implements BinaryTree<E> {
protected BTNodo<E> raiz;
protected int size;

public ArbolBinarioEnlazado() { raiz = null; size =0; }

public int size() { return size; }                    T_size(n) → O(1)

public boolean hasLeft( Position<E> v) {
        BTNodo<E> n = checkPosition(v); // Propaga excepción de pos. inválida
        return n.getLeft() != null;                   T_hasLeft(n) → O(1)
}
```

Implementación de árbol binario

```
public class ArbolBinarioEnlazado<E> implements BinaryTree<E> {
protected BTNodo<E> raiz;
protected int size;

public ArbolBinarioEnlazado() { raiz = null; size =0; }

public int size() { return size; }                    T_size(n) → O(1)

public boolean hasLeft( Position<E> v) {
        BTNodo<E> n = checkPosition(v); // Propaga excepción de pos. inválida
        return n.getLeft() != null;                   T_hasLeft(n) → O(1)
}
public Position<E> left(Position<E> v) {
        BTNodo<E> n = checkPosition(v); // Propaga excepción pos. inválida
        if( n.getLeft() == null ) throw new BoundaryViolationException( "..." );
        return n.getLeft();                           T_left(n) → O(1)
}
```

Implementación de árbol binario

```
public class ArbolBinarioEnlazado<E> implements BinaryTree<E> {
protected BTNodo<E> raiz;
protected int size;

public ArbolBinarioEnlazado() { raiz = null; size =0; }

public int size() { return size; }                    T_size(n) → O(1)

public boolean hasLeft( Position<E> v) {
        BTNodo<E> n = checkPosition(v); // Propaga excepción de pos. inválida
        return n.getLeft() != null;                   T_hasLeft(n) → O(1)
}
public Position<E> left(Position<E> v) {
        BTNodo<E> n = checkPosition(v); // Propaga excepción pos. inválida
        if( n.getLeft() == null ) throw new BoundaryViolationException( "..." );
        return n.getLeft();                           T_left(n) → O(1)
}
public boolean isInternal(Position<E> v) {
        return hasLeft(v) || hasRight(v); // Propaga excepción de pos. inválida
}                                                     T_isInternal(n) → O(1)
```

Implementación de árbol binario

```
public Iterable<Position<E>> children( Position<E> v ) throws InvalidPositionException {
        PositionList<Position<E>> hijos = new ListaDoblementeEnlazada<Position<E>>();
        if( hasLeft(v) )
                hijos.addLast( left(v) );
        if( hasRight(v) )
                hijos.addLast( right(v) );
        return hijos;
}
```

$T_{children}(n) \rightarrow O(1)$

Como tarea

- createRoot(e): Agrega un nodo raíz con rótulo e. Error si ya hay raíz
- addLeft(v, e): Crea un nodo w hijo izquierdo de v con rótulo e, error si v ya tiene hijo izquierdo
- addRight(v,e): Crea un nodo w hijo derecho de v con rótulo e, error si v ya tiene hijo derecho
- remove(v): Elimina el nodo v (si v tiene un hijo, reemplaza a v por su hijo, si v tiene dos hijos entonces error)
- attach(v, T1, T2): Setea T1 como hijo izq de v y T2 como hijo derecho de v (error si v no es hoja)

Ver fragmento de código 7.16 a 7.20 en páginas 406 a 412 (implementación de LinkedBinaryTree)

Cuidado!

```
public interface BinaryTree<E> extends Tree<E>
```

Esto hace que también tengamos que implementar todos los métodos de árboles generales (Tree)

- addBefore
  - debe chequear que se intenta agregar un hijo izquierdo antes de un derecho
- addAfter
  - debe chequear que se intenta agregar un hijo derecho antes de un izquierdo
- addFirstChild/addLastChild
  - similar a anterior
- removeInternalNode
  - debe chequear que el nodo a eliminar tenga 1 hijo, y si tiene dos, lanzar excepción

Árboles Binarios - Recorridos

Algoritmo preorden(T, v)

```
Visitar v
si T.hasLeft( v ) entonces
        preorden( T, T.left(v) )
si T.hasRight( v ) entonces
        preorden( T, T.right(v) )
```

Algoritmo posorden(T, v)

```
si T.hasLeft( v ) entonces
        posorden( T, T.left(v) )
si T.hasRight( v ) entonces
        posorden( T, T.right(v) )
Visitar v
```

Algoritmo Inorden(T, v)

```
si T.hasLeft( v ) entonces
        Inorden( T, T.left(v) )
Visitar v
si T.hasRight( v ) entonces
        Inorden( T, T.right(v) )
```

Árboles Binarios con Arreglos

La componente 0 no se usa

La raíz se almacena en la componente 1

El hijo izquierdo de la componente i se almacena en la componente 2*i

El hijo derecho de la componente i se almacena en la componente 2*i + 1

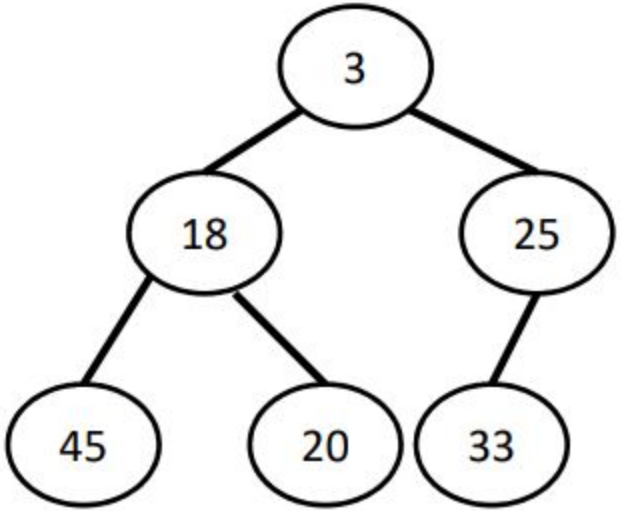
El padre de i está en la componente i / 2

Nota: Puede haber componentes vacías si el árbol no está lleno (un árbol está lleno si tiene todos los nodos; un árbol binario lleno de altura h tiene 2^{h+1}-1 nodos).

Hijo_izquierdo(i) = 2i

Hijo_derecho(i) = 2i+1

Padre(i) = i div 2



0	1	2	3	4	5	6	7	8	9	10	11	12	13
	3	18	25	45	20	33							

Árboles Binarios con Arreglos

Esta implementación es muy útil para implementar un tipo de árbol binario llamado heap que veremos más adelante.

Bibliografía

Goodrich & Tamassia, Data Structures and Algorithms in Java, 4th Edition, capítulo 7.

THE END.